PID 控制算法简述

在嵌入式系统中，我们经常需要控制某个物理量，使其达到或保持在一个期望值。

比如，控制一个电机的转速、一个加热器的温度、一个飞行器的姿态等。如果仅仅是开环控制（即根据输入直接输出，不考虑实际效果），一旦系统受到干扰或模型不准确，就很难达到精确控制的目的。

这时候，就需要**反馈控制**。

反馈控制的核心思想是：将系统的实际输出与期望输出进行比较，得到一个误差，然后根据这个误差来调整控制器的输出，从而使系统输出逐渐趋近于期望值。

PID控制算法（Proportional-Integral-Derivative Control）是工业界应用最广泛、最经典的一种反馈控制算法。

它通过计算**比例（P）**、**积分（I）** 和**微分（D）** 三个部分的组合来产生控制输出。

1 P、I、D 的含义

P 的含义

P-Propotional，比例项。

P 是按比例 K_p 缩放当前时刻的**误差大小**来起到控制作用的。那么首先定义“当前时刻的误差”：

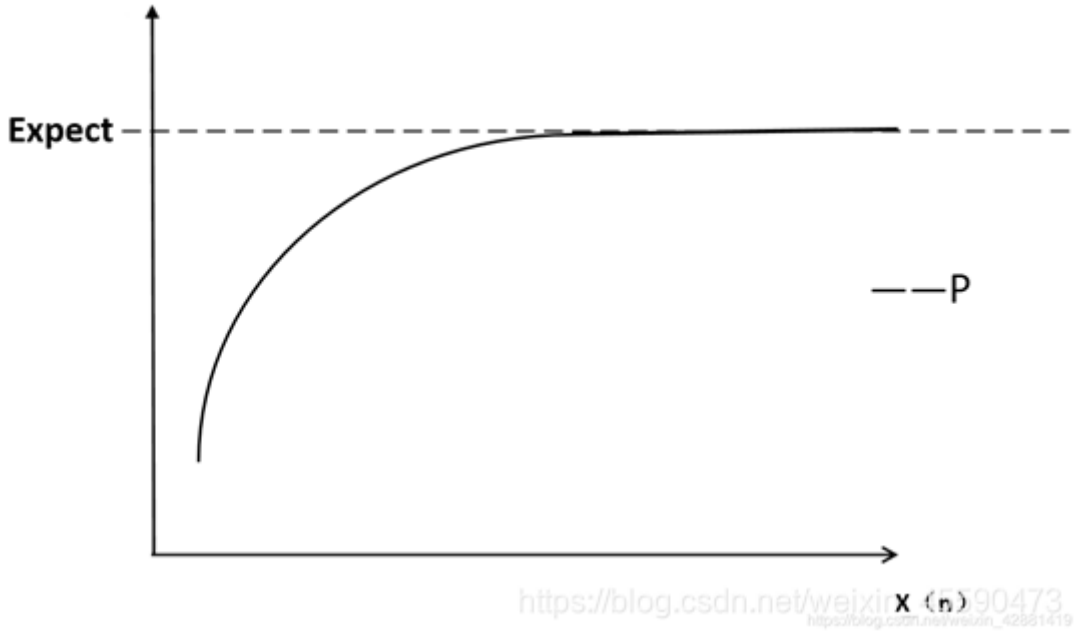
假比当前时刻为 t 时刻，则误差为 $e(t) = \mathrm{Expect} - X_t$ ，其中：

- X_t ： t 时刻的系统的输出值
- $Expect$ ： 期望值

进一步地，P 环节的输出 $u_P(t)$ 为

$$u_P(t) = K_p \cdot e(t) \tag{1}$$

可以模拟成下图：



https://blog.csdn.net/weixin_45590473

不难理解，P 环节在误差大时能快速反应，但是最终仍无法完全拟合到 $Expect$ 值（称为“静差”）。

于是就有了 I 环节。

I 的含义

I-Integral，积分项。

积分项的作用是**消除静差**。它根据**过去所有误差的累积**来产生控制作用。

如果系统长时间存在小误差，即使比例项无法完全消除，积分项也会随着时间的推移而逐渐积累，最终产生足够大的控制作用来消除这个误差。

关于 t 时刻的误差 $e(t) = \text{Expect} - X_t$ 的序列 $e_1, e_2, ..., e_t$ 的和（或有的设计成后 k 个误差项求和）乘以比例系数 K_i 起控制作用。

但一般都写成积分项：

$$u_i(t) = K_i \int_0^t e(\tau) d\tau \tag{2}$$

其实就是 $u_I(t) = K_i \cdot \sum_0^t e(\tau) d\tau$ ，或者写成离散形式： $I_k = I_{k-1} + K_i \cdot e(t)$ 。

但是仔细想想就会觉得有问题，如果我一开始误差比较大，然后终于借助 I 环节消除了静差，结果前面误差还在，下一次 I 环节直接就又把输出推超过 $Expect$ 了。这个问题在设定值正好是器械参数最大

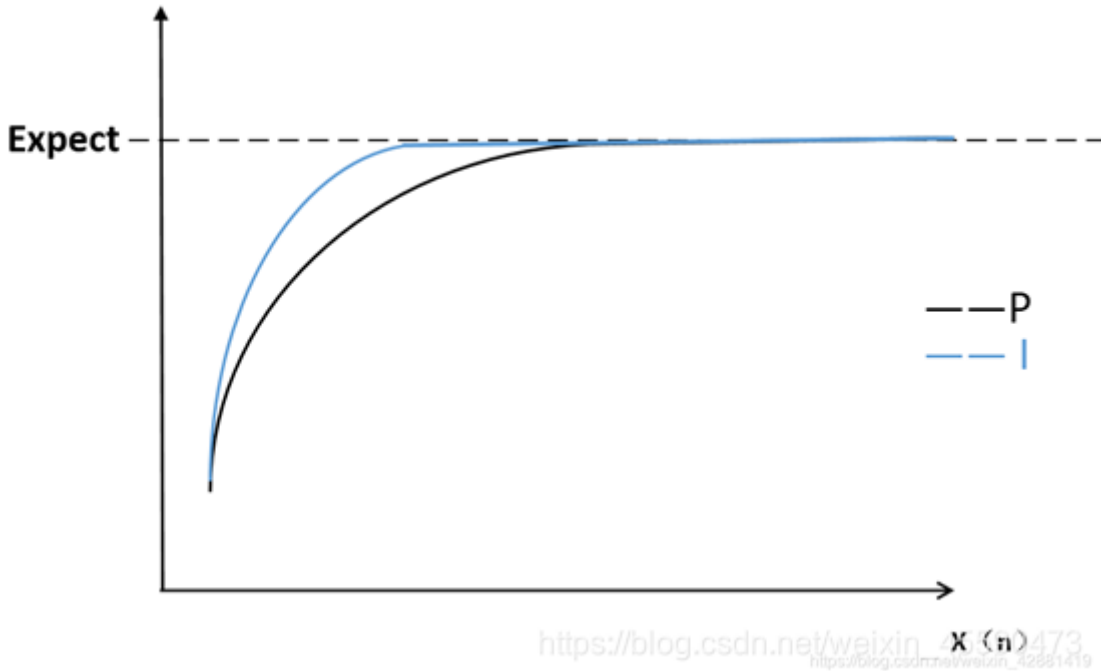
值时会造成更不好的后果。这叫**积分饱和**。

还有**积分分离**问题（I 项使输出值变化太快结果硬件没跟上）等，但都不及**积分饱和**之主要与严重。

主要的解决措施：

1. 在输出达到 $Expect$ 时，停止积分项的累积；
2. 在输出达到 $Expect$ 时，计算一个反向的积分项，使其向未饱和方向回落，直到输出不再饱和；
3. 只有当误差在一定范围内时才进行积分；
4. 在系统启动阶段、设定值大幅改变时或者误差超过某个阈值时，暂时关闭积分（$K_i = 0$ 之类的），待误差进入指定范围后再开启积分。

总的来说，I 项仍然是消除静差的必不可少的选项。但在可以接受微小静差的工程环境下，也可以直接采用把 K_i 调小点的措施来减少其影响。



https://blog.csdn.net/weixin_42891419

D 的含义

D-Derivative，微分项。

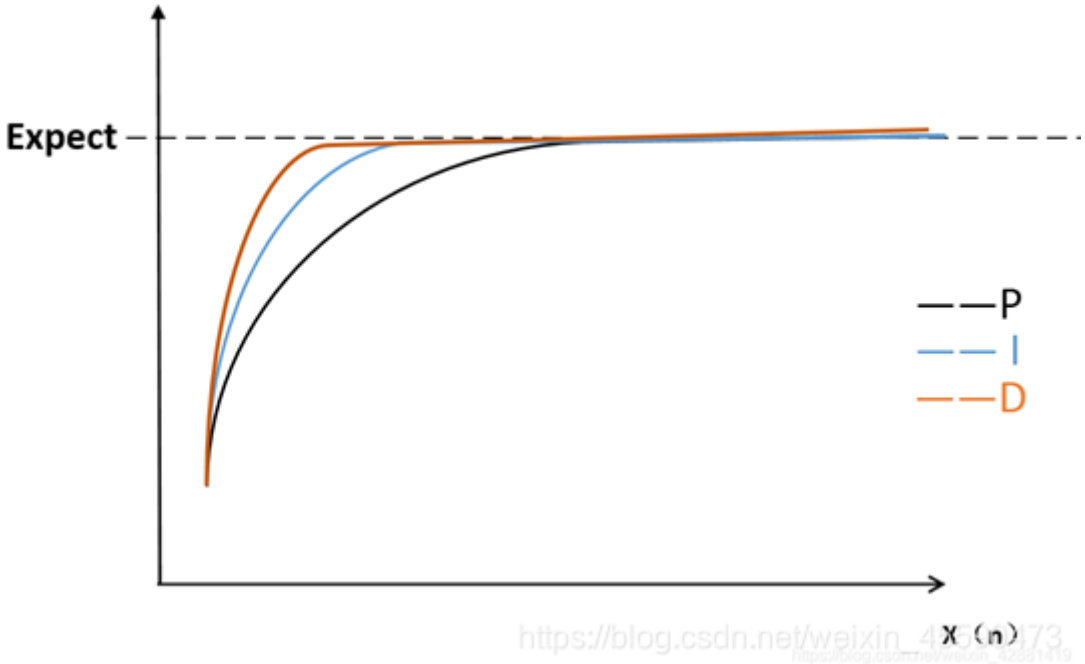
微分项的作用是预测误差变化趋势，并提前进行修正。

具体来说，它根据当前误差的**变化率**来产生控制作用。公式为:

$$u_D(t) = K_D \cdot \frac{de(t)}{dt} \tag{3}$$

如果误差正在快速增大，微分项为正，增大 PID 最终输出，以加速到达期望值；如果误差正在快速减小，微分项又会为负值。

简单来说，有点类似于一个阻尼器的作用。



PID 算法

最终合并的输出为：

$$u(t) = u_P(t) + u_I(t) + u_D(t) = K_p \cdot e(t) + K_i \int_0^t e(\tau)d\tau + K_D \cdot \frac{de(t)}{dt} \tag{4}$$

那么这个 PID 最终的输出 $u(t)$ 怎么作用到真正的调控过程呢？这涉及到两种方式：位置式 PID 和增量式 PID。

2 位置式 PID 和增量式 PID

简单地说，位置式PID就是直接把 $u(t)$ 作为输出值，比如（概念上）直接作为电机占空比之类的。增量式PID是把输出作为上一次的增量。

无论哪种实现方式，PID控制器都依赖于三个基本组成部分对误差信号 $e(t)$ 进行处理。

在嵌入式系统中，由于是离散采样和计算，PID算法需要进行离散化处理。假设采样周期为 Δt，当前时刻为 t，前一时刻为 $t-1$，$e(t)$ 为当前时刻误差，那么：

- **P项离散化：** $u_P(t) = K_p \cdot e(t)$
- **I项离散化：** $u_I(t) = I_{t-1} + K_i \cdot e(t) \cdot \Delta t$
- **D项离散化：** $u_D(t) = K_d \cdot \dfrac{e(t) - e(t-1)}{\Delta t}$

我们以多旋翼无人机的高度控制为例，

目标设定为让无人机从地面上升并保持在指定高度（如 10 米）。

以无人机四个螺旋桨为具体设备，被PID调控的数据为电机平均转速（通过PWM信号控制）。四个螺旋桨的平均转速越高，无人机产生的升力越大，高度就越高。

PID控制器根据当前高度与目标高度的差，计算出应该的平均转速，从而让无人机稳定上升。

> 虽然这个是不符合实际的，因为其实我们是遥控器控制它上升下降而不是“指定高度”。
>
> PID 真正作用在姿态控制上。比如现在飞机是悬停，也就是 0 度的俯仰角。我要前进，即“调整为20度”的指令。那么这 20-0 = 20度，就是误差，经过 PID，输出前后电机的占空比，从而实现向前飞。

假设条件：

- 我们只考虑垂直方向的运动，不考虑水平移动和姿态。
- PWM占空比范围是 0-100%（对应电机转速 0 到最大转速）。
- 存在一个维持无人机悬停所需的最小PWM占空比（例如 40%）。

位置式 PID

计算出的 U_k 直接代表当前时刻所有螺旋桨的**平均PWM占空比指令**（一个0-100%的绝对值）。

公式依旧是 $u(t) = K_p e_t + K_i \sum_{i=0}^{t} e_i \Delta t + K_d \frac{e_t - e_{t-1}}{\Delta t}$ ，

其中 e_t ： 当前高度误差 = 目标高度 - 当前实际高度。

工作流程示例

目标高度 = 10m。

无人机在地面。t=0s (初始状态)：

- 当前高度 0m
- $e_0 = 10 - 0 = 10\text{m}$ (误差很大)
- PID计算 $u(0)$： $u_P(0)$ 很大 ($K_p \times 10$)，I项为0，D项为0。假设 $u_P(0)$ 计算出来是 80（对应 80% 占空比）

- 微控制器立即设置所有螺旋桨的平均PWM占空比为 80%。电机开始高速旋转，产生巨大升力，无人机快速上升

无人机上升中，比如 t=1s：

- 当前高度 3m
- $e_1 = 10 - 3 = 7\text{m}$ (误差变小)
- PID计算 $u(1)$：P项 ($K_p \times 7$) 变小。I项开始累积 ($I_1 = I_0 + K_i \times e_1 \times \Delta t$)。D项可能为负值（因为高度正在快速增加，误差正在快速减小，D项会起到抑制作用）。假设 $u(1)$ 计算出来是 70。
- PWM占空比降低到 70%。无人机继续上升，但升力有所减小。

无人机接近目标高度，比如 t=8s：

- 当前高度 = 9.8m。
- $e_3 = 10 - 9.8 = 0.2\text{m}$ (误差很小)。
- PID计算 $u(8)$：P项 ($K_p \times 0.2$) 变得非常小。I项已经累积了之前的大量误差，可能已经达到一个较高的值。D项可能接近0（如果速度变化不大）。假设 $u(8)$ 计算出来是 42。
- PWM占空比降低到 42%。无人机继续微调高度。

无人机稳定在目标高度，比如 t=9s：

- 当前高度 = 10m。
- $e_5 = 10 - 10 = 0\text{m}$ (误差为零)。
- PID计算 $u(9)$：
  - P项：$K_p \times 0 = 0$。
  - D项：$K_d \times \frac{0 - 0}{\Delta t} = 0$ (如果高度不再变化)。
  - **I项：** 此时，积分项 $\sum e_i \Delta t$ 已经累积到一个特定的值，这个值正好能够补偿重力，**使** $u(9)$ **维持在无人机悬停所需的占空比（例如 40%）**。
- PWM占空比保持在 40%。无人机稳定悬停在 10m 高度。

位置式PID的最后就是误差为零，积分项维持一个非零的输出 $u_I(t)$，这个输出就是维持系统稳定所需的“基准力”。同时此时的 $u(t)$ 是**绝对的**PWM占空比。

积分饱和问题示例：

如果无人机被外力强行按到（例如被障碍物卡住），无法上升。即使PID计算出 100% 占空比，它也无法上升。但此时误差依然很大 (10m)。积分项会持续累积，可能累积到相当于 200% 占空比的值。

或当无人机脱离，这个超大的积分项导致无人机以 100% 占空比猛冲向上，冲过 10m 很多，然后才缓慢降落，产生严重过冲。

增量式 PID

计算出的是当前时刻 PWM 占空比相对于上一时刻的变化量 $\Delta u(t)$ 。

最终的占空比是 $u(t-1)+\Delta u(t)$ 。

数学表达式为 $\Delta u(t)=K_p(e_t-e_{t-1})+K_ie_t\Delta t+K_d\dfrac{e_t-2e_{t-1}+e_{t-2}}{\Delta t}$

工作流程示例：

目标高度 = 10m。

无人机在地面。t=0s (初始状态)：

- 当前高度 = 0m。
- $e(0)=10-0=10\text{m}$ (误差很大)。
- 假设无人机初始PWM占空比 $u(-\Delta t)$ 为 0%（电机未启动）。
- PID计算 $\Delta u(0)$ ：

  $P_{\text{增量}}$ 很大 ($K_p\times(e(0)-e(-\Delta t))=K_p\times(10-0)$)，

  $I_{\text{增量}}$ 很大 ($K_i\times e(0)\times\Delta t=K_i\times10\times\Delta t$)。

  D项为0（ $e(-\Delta t)$ 和 $e(-2\Delta t)$ 也视为0）。

  假设 $\Delta u(0)$ 计算出来是 80。
- 更新 $u(0)$ ： $u(0)=u(-\Delta t)+\Delta u(0)=0+80=80$ 。
- 微控制器立即设置所有螺旋桨的平均PWM占空比为 80%。电机开始高速旋转，产生巨大升力，无人机快速上升。

无人机上升中，比如 t=1s：

- 当前高度 = 3m。

- $e(1) = 10 - 3 = 7\text{m}$ (误差变小)。
- PID计算 $\Delta u(1)$：

  $P_{\text{增量}}$ $(e(1) - e(0)) = (7 - 10) = -3$ （P项增量为负，因为误差在减小）。

  $I_{\text{增量}}$ ($K_i \times e(1) \times \Delta t = K_i \times 7 \times \Delta t$) 仍为正。

  D项可能会是负值（因为误差在快速减小）。

  假设 $\Delta u(1)$ 计算出来是 -10。
- 更新 $u(1)$： $u(1) = u(0) + \Delta u(1) = 80 + (-10) = 70$ 。
- PWM占空比降低到 70%。无人机继续上升，但升力有所减小。

无人机接近目标高度，比如 t=8s

- 当前高度 = 9.8m。
- $e(8) = 10 - 9.8 = 0.2\text{m}$ (误差很小)。
- PID计算 $\Delta u(8)$：

  $P_{\text{增量}}$ 很小。

  $I_{\text{增量}}$ ($K_i \times e(8) \times \Delta t = K_i \times 0.2 \times \Delta t$) 也很小。

  D项接近0。

  假设 $\Delta u(8)$ 计算出来是 2。
- 更新 $u(8)$： $u(8) = u(7) + \Delta u(8) = 40 + 2 = 42$ (假设上一步 $u(7)$ 是 40)。
- PWM占空比变为 42%。无人机继续微调高度。

无人机稳定在目标高度，比如 t=9s：

- 当前高度 = 10m。
- $e(9) = 10 - 10 = 0\text{m}$ (误差为零)。

  $e(9 - \Delta t) = 0\text{m}$ (上一个时刻误差也为0)。

  $e(9 - 2\Delta t) = 0\text{m}$ (上上一个时刻误差也为0)。

  > 这里说前几个时刻的误差都为0不是说随便假设的，而是说前几个时刻误差都为0了，此时 Δu 也终于为 0 了。然后此时假设为 $t = 9$ 时。
- PID计算 $\Delta u(9)$：

  $P_{\text{增量}}$: $K_p(e(9) - e(9 - \Delta t)) = K_p(0 - 0) = 0$ 。

  $I_{\text{增量}}$: $K_i \times e(9) \times \Delta t = K_i \times 0 \times \Delta t = 0$ 。

$D_{\text{增量}}$：$K_d \times \frac{e(9) - 2e(9-\Delta t) + e(9-2\Delta t)}{\Delta t} = K_d \times \frac{0 - 2 \times 0 + 0}{\Delta t} = 0$。

所以，$\Delta u(9) = 0$。

- 更新 $u(9)$：$u(9) = u(9-\Delta t) + \Delta u(9) = u(9-\Delta t) + 0 = u(9-\Delta t)$。
- PWM占空比保持不变。它会保持在无人机悬停所需的占空比（例如 40%），因为前一个时刻 $u(9-\Delta t)$ 已经达到了这个值。无人机稳定悬停在 10m 高度。

增量式PID的关键点： 最终的PWM占空比 $u(t)$ 是通过在 $u(t-\Delta t)$ 的基础上叠加增量 $\Delta u(t)$ 而得到的。当误差稳定为零时，$\Delta u(t)$ 也为零，使 $u(t)$ 保持不变。

3 实践中 Kp Ki Kd 的设置与调参

实践中

简单地说，PID 在四轴中的应用在姿态上。

比如现在飞机是悬停，也就是 0 度的俯仰角。我要前进，即“调整为 20度”的指令。那么这 20-0 = 20 度，就是误差，经过 PID，输出前后电机的占空比，从而实现向前飞。

具体地说，这分为两个环节，也就是**级联PID**。

内环：

- 角速度环。
- 输入 (Setpoint)： 期望的角速度 (例如，“以50度/秒的速度向右滚转”)。这个指令来自于外环。
- 陀螺仪实时测量当前实际角速度。
- 输出： 直接的电机PWM调整指令。它会精确计算四个电机应该如何差速运转，以产生期望的力矩，从而达到期望的角速度。

外环：

- 角度环 。
- 输入 (Setpoint)： 期望的角度 (例如，“向右倾斜20度”)。这个指令来自于你的遥控器。
- 反馈： 加速度计和陀螺仪融合解算出的当前实际角度。
- 输出： 给内环下达的一个指令，即一个期望的角速度。

> 比如这种：目标是倾斜20度，现在只倾斜了15度，还差5度。根据P、I、D计算，为了最快最稳地消除这5度误差，我应该命令‘内环’ 以50度/秒的速度先转起来。”

调参

PID 调参（PID tuning），又称**整定**，是一个重要的环节。

永远先调内环，再调外环！

调内环 (Rate Loop): 先将飞控设置为“手动模式”，遥控器指令直接控制期望角速度。内环的PID整定的目标是让无人机对指令响应快而精确，并且在快速翻滚后能立刻定住，没有多余的晃动或反弹。

调外环 (Angle Loop): 调整外环的P值（通常外环只需要P控制器），目标是让无人机在你松开遥控杆后，能够快速、平稳地自动回平，没有过冲和振荡。

阅读相关资料时提到，PID 算法的由来是由 P控制、PI 控制、PD 控制最终演变为 PID控制而来的。这是因为“它集三者之长：既有比例作用的及时迅速，又有积分作用的消除余差能力，还有微分作用的超前控制功能。”

鉴于此，在调参时，我们也可以有选择性地，根据自身需求地调：

- 更注重快速响应 ⇒ PI 更重要
- 更注重系统稳定 ⇒ PD 更重要
- PID 都挺重要

也可以在开始不用这么清晰，根据实际表现来调也是一个不错的方法。

对于**较为良好的系统**（线性、可开环等）通常有两种方式调参：

1. 基于模型（model）的方法
2. 基于硬件实验的方法

基于模型的方法

模型（model），书面地说，是一个数学表示，它描述了被控物理系统（Plant）的动态关系，即系统的输入如何随时间影响其输出。

也可以简单理解为一种“模拟”。

在调节时，P 的最初选取一般有两种情况：

1. **P 偏小一点**：稳定时，实际值在目标值之下，一直存在误差，这时再从0开始，一直加大I，消除稳定时的误差，这种情况下的最终稳定曲线会一直保持在目标曲线之下，达到比较稳定的调节效果，不会有过冲
2. **P 大一点**：在第一次到达目标值的时候有一定的过冲，但之后就会稳定，其与第一种相比反应速度更快

同时，D 的作用在**大惯性系统**中起到较为重要的作用，加上合适的D之后，实际曲线到达目标位置的时间更短，过冲幅度也会降低一些。

但是 D 调节的过大之后，反倒会放大系统趋势的影响，使系统出现震荡，难以稳定。

这里简单地用 python 模拟这个过程：（展示）

> 代码场景 1：
>
> 无人机质量（mass）为 3kg、g 取 9.8 m/s^2、空气阻力系数取 0.5 $N \cdot s^2/m^2$、初始高度 0、目标高度 10。
>
> 图中实线为电机转速（由推力体现），虚线为无人机高度。
>
> 代码场景 2：
>
> 转动惯量（inertia）为 0.1、空气阻力系数取 0.5 $N \cdot s^2/m^2$、初始角度 0、目标角度 30。
>
> 左图为外环 PID，右图为内环 PID。

基于硬件实验的办法

当然，这种方法可以非常直观的通过仪器（四轴无人机）的表现来反复试错调参，但同时，我们也有更系统的方法。

启发式方法：

这种方法为旧时数字模拟时代的方法，可以了解一下。

这种更系统的方法可以描述如下：

1. **实验驱动：** 首先通过实验（运行输入序列）来获取系统响应数据。
2. **启发式初步整定：** 利用Ziegler-Nichols或Cohen-Coon等经验方法，从实验数据中提取信息，计算出PID参数的**起始点**。
3. **迭代优化：** 在此基础上，进行**手动微调**和反复测试，以达到最佳性能。这是一个持续优化的过程。

我们这里关注于 Ziegler-Nichols (Z-N)方法。它的核心思想是：

通过简单的实验来摸清被控对象的动态特性，然后根据特性查表或套用公式，得到一组不错的初始PID参数。

Z-N方法主要分为两种，适用于不同的场景：

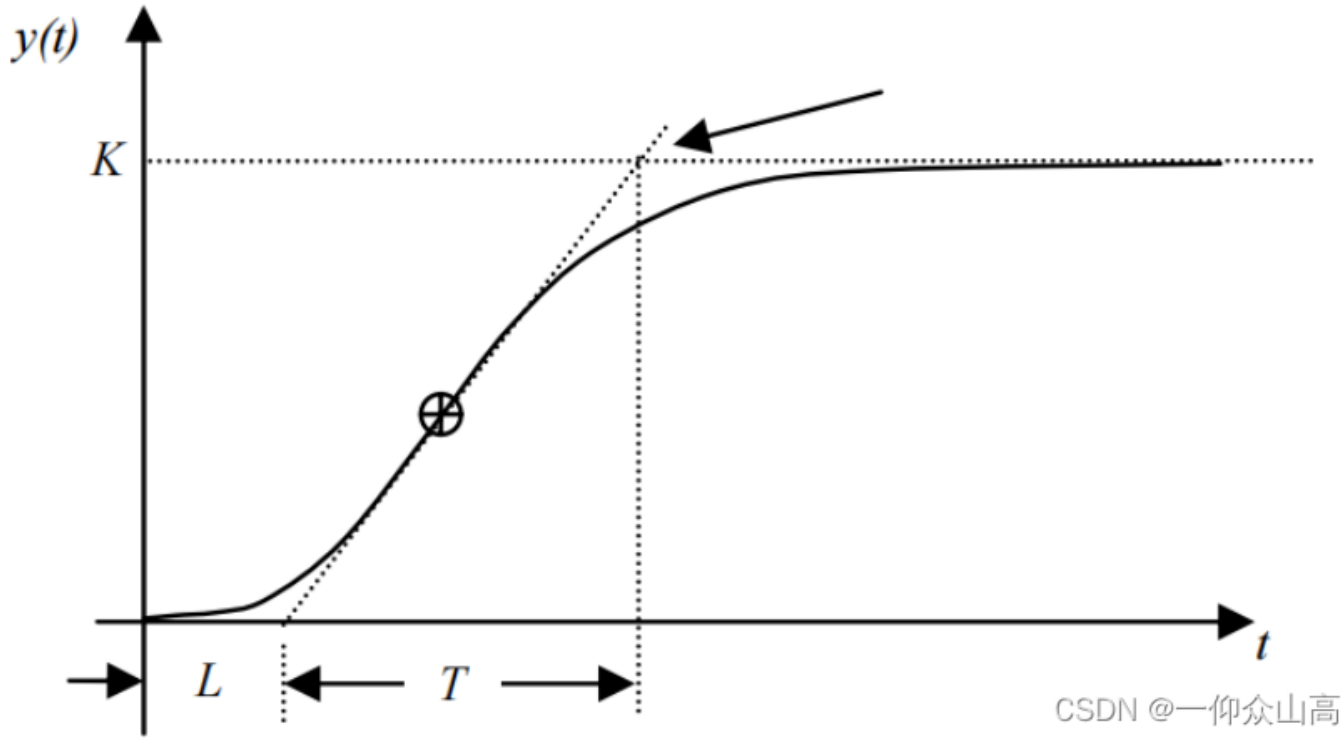
方法一：过程反应曲线法 (Process Reaction Curve Method)

这种方法也称为**阶跃响应法** 或**第一方法** 。

适用场景：被控对象在开环状态下是稳定的（即你给它一个固定的输入，它的输出最终会稳定在一个新的值，而不会发散）。

实验步骤：

1. 使用手动模式（而非PID控制）
2. **施加阶跃输入：** 给执行器一个阶跃输入。例如，对于无人机，将PWM占空比从一个稳定的悬停值（比如40%）瞬间跳变到一个更高的值（比如60%）
3. **记录S形曲线：** 观察并记录系统的输出（例如无人机的垂直加速度）随时间的变化。得到一条类似“S”形的响应曲线。



CSDN @一仰众山高

4. **测量三个关键参数：**
   - K (过程增益 / Process Gain): 系统对输入的敏感程度。
     $$K = \frac{\Delta \text{Output}}{\Delta \text{Input}} = \frac{\text{输出的最终稳定变化量}}{\text{输入的跳变量}}$$
   - L (延迟时间 / Lag Time): 从输入变化开始，到输出开始响应之间的时间。
   - T (时间常数 / Time Constant): 输出开始响应后，达到最终变化量63.2%所需的时间。在S形曲线上，一个简便的作图法是：在曲线斜率最大的点（拐点）画一条切线，切线与时间轴的交点到L点的距离就是T。

计算PID参数：

测量出K, L, T之后，就可以根据下表来计算PID参数的初始值：

控制器类型	Kp	Ti (积分时间)	Td (微分时间)
P	`T / (K * L)`	-	-
PI	`0.9 * T / (K * L)`	`L / 0.3`	-
PID	`1.2 * T / (K * L)`	`2 * L`	`0.5 * L`

表格给出的是积分时间 `Ti` 和微分时间 `Td`。需要将它们转换为我们常用的积分增益 `Ki` 和微分增益 `Kd` （即参数）：

- $K_i = K_p / T_i$
- $K_d = K_p \cdot T_d$

方法二：临界振荡法 (Critical Oscillation Method)

这种方法也称为**临界比例增益法** 或**第二方法** 。

适用场景：

- 系统可以在**闭环** 状态下（反馈控制下）安全地进行测试。
- 系统在纯比例控制下（P 模式下），能够产生持续的等幅振荡。

> 当逐渐增加Kp时，无人机的行为会经历以下阶段：
>
> 1. Kp很小： 无人机反应迟钝。
> 2. Kp适中： 无人机响应变得迅速。
> 3. Kp接近临界点： 在受到一个小的扰动后，无人机会出现持续几秒钟的、幅度几乎不衰减的高频抖动或“颤振”。这个抖动的幅度可能不是完全恒定，可能会轻微地“呼吸”，但它既没有快速消失，也没有快速放大失控。—— 这就是“临界振荡”
> 4. Kp过大： 振荡会变得发散，幅度越来越大，无人机失控。

实验步骤：

1. Ki和Kd设为0。
2. **从小到大增加Kp：** 将无人机起飞并置于闭环控制下。从一个非常小的Kp值开始慢慢增加Kp。
3. **找到临界点：** 继续增加Kp，直到系统输出（无人机的高度）出现**临界振荡**
4. **记录两个关键参数：**
   - K_{cu} (临界比例增益 / Critical Gain): 导致系统产生临界振荡的那个Kp值。

- T_u (临界振荡周期 / Ultimate Period): 在临界振荡状态下，一个完整的振荡周期所需要的时间（从一个波峰到下一个波峰的时间）。

临界震荡?

本来是指等幅振荡，但是无人机因为一系列现实因素，就算依靠P环节也无法达到完美的等幅振荡。一般观测以下几种现象：

- 现象A：持续的“颤振” (Jitter / Shaking)
  - 无人机悬停时，整体看起来很“紧张”，能听到电机发出高频的“嗡嗡”声，机臂末端有肉眼可见的、小幅度的、快速的抖动。这种抖动不会自行消失，也不会迅速恶化。
  - 这通常是整定**内环（角速度环）**时寻找的临界点。
- 现象B：受扰后的持续摆动 (Sustained Oscillation after Disturbance)
  - 快速地打一下横滚或俯仰杆然后立刻回中，无人机会开始来回摆动。这个摆动的幅度在几个周期内几乎没有明显的衰减。它不像一个调得好的无人机那样“一下”就定住了，而是像一个钟摆一样“晃”好几下。
  - 这通常是整定**外环（角度环）**时寻找的临界点。

计算PID参数：

记录下 K_{cu} 和 T_u 之后，根据下表计算PID参数的初始值：

控制器类型	Kp	Ti (积分时间)	Td (微分时间)
P	`0.5 * K_cu`	-	-
PI	`0.45 * K_cu`	`T_u / 1.2`	-
PID	`0.6 * K_cu`	`T_u / 2`	`T_u / 8`

同样， $K_i = K_p/T_i$ 和 $K_d = K_p \cdot T_d$ 。

对于无人机，临界振荡法更常用，但也更危险，务必在安全环境下进行。

现代灰箱方法

黑匣子频率响应分析法是一种先进的、基于实验数据的飞行控制器参数整定方法。它将时域的飞行数据转换到频域进行分析，从而实现对滤波器和PID控制器的精确、可视化和科学化的优化。

其核心工作流程可以概括为“先降噪，再增益”：

1. **数据采集 (硬件实验):**
    - 通过实际飞行，特别是执行一系列快速的机动动作，来激励无人机系统。
    - 飞控板上的“黑匣-子”功能会以高频率（如2kHz）记录下陀螺仪的传感器读数、遥-控器指令和电机输出等所有关键数据。
2. **频域分析 (模型分析):**
    - 将采集到的日志数据导入专门的分析软件（如PID Toolbox）。
    - 软件通过**快速傅里叶变换 (FFT)**，将时域的陀螺仪信号转换为**功率谱密度图**。
    - 这张图以**频率 (Hz)** 为横轴，揭示了信号在不同频率下的“能量”分布。工程师可以清晰地看到由**电机振动**等物理因素产生的**噪声峰值 (Noise Peaks)**。
3. **滤波器整定 (降噪):**
    - 调参的第一步**不是**动PID，而是配置飞控的**数字滤波器**（如低通滤波器和陷波滤波器）。
    - 目标是精确地“切掉”频谱图上看到的噪声峰值，得到一条干净的传感器信号曲线，同时要避免引入过多的**信号延迟 (Delay)**。
4. **PID整定 (增益):**
    - 在已经滤除噪声的基础上，工程师可以更安全、更激进地提高**PID增益**。
    - 通过分析系统的**频率响应（波特图）**，可以科学地调整P和D增益的比例，以在不牺牲稳定性的前提下，最大化无人机的**响应带宽**（即“跟手程度”）。